

A threshold-gated reduction from Hitting Set to BCNF violation

Working manuscript for expert review
2026-09-23

Abstract

We give a deterministic polynomial-time reduction from cardinality-bounded Hitting Set to the search problem of finding a Boyce–Codd normal form violation in a specified attribute subset. A Horn threshold circuit records whether a selected set exceeds the binary bound while its auxiliary attributes remain outside the tested relation. A target violation exists exactly when a bounded hitting set exists, and every valid target output decodes correctly, including NO-SOLUTION. The construction uses at most quadratic many attributes and dependencies. Earlier hardness results are known; we claim a complete explicit rule rather than a new complexity classification or historical priority.

1. Introduction

A Boyce–Codd normal form (BCNF) violation is a determinant whose closure gains an attribute of a relation but misses another. The closure may traverse functional dependencies (FDs) on a larger attribute set. Finding such a partial closure is the projected BCNF problem studied here.

Bernstein and Beeri’s accessible 1976 report reduces an exact-one-per-set hitting variant without a binary cardinality bound [1, Part III §2, Theorem 8]. Vardi reproduces that construction [2, Chapter 5, Theorem 5.9]. Garey and Johnson list a Vertex Cover transformation for the same BCNF predicate [3, Appendix A4, SR29]. The full 1979 Beeri–Bernstein journal construction was unavailable for comparison [4]. We give a direct bounded-Hitting-Set rule, including recovery from every valid target answer. The construction uses a polynomial Horn threshold circuit to suppress violations from oversized selections.

2. Problems and reduction contract

A Hitting Set input is an explicitly listed universe $U = (u_0, \dots, u_{n-1})$, subsets $E_0, \dots, E_{m-1}$ of U , and a nonnegative binary integer k . A positive output lists at most k members meeting every E_j . Otherwise the only valid output is NO-SOLUTION. The executable encoding admits distinct string or integer element labels; the input list fixes their order.

A target input consists of an attribute set A , FDs F over A , and a tested subset $R \subseteq A$. Write X^+ for FD closure. A positive output is (X, y, z) with $X \subseteq R$, distinct $y, z \in R - X$, $y \in X^+$, and $z \notin X^+$. Otherwise the only valid output is NO-SOLUTION. A solver may choose any valid triple.

The forward map F_{fwd} produces a legal target instance. The decoder G receives the original source instance and a target output. Its obligation is

$$\forall x \forall w \in S_{\text{BCNF}(F_{\text{fwd}}(x))} : G(x, w) \in S_{\text{HS}(x)}. \quad (1)$$

3. Construction

For each position i in the input universe, create an attribute a_i representing u_i . For each set E_j , create an auxiliary attribute b_j . Add markers C, D . The tested relation contains all a_i together with C, D ; every b_j is outside it. Add $a_i \rightarrow b_j$ precisely when $u_i \in E_j$, and add $(b_0, \dots, b_{m-1}) \rightarrow C$. For an empty family, the latter FD has an empty left side.

When $k < n$, set $q = k + 1$. For $1 \leq i \leq n$ and $1 \leq j \leq \min(i, q)$, create an auxiliary attribute $t_{i,j}$ outside the tested relation. The threshold FDs are

$$a_{i-1} \rightarrow t_{i,1} \quad (1 \leq i \leq n), \quad (2)$$

$$t_{i-1,j} \rightarrow t_{i,j} \quad (1 \leq j \leq \min(i-1, q)), \quad (3)$$

$$(t_{i-1,j-1}, a_{i-1}) \rightarrow t_{i,j} \quad (2 \leq j \leq \min(i, q)). \quad (4)$$

Add $(C, t_{n,q}) \rightarrow D$. When $k \geq n$, omit the threshold circuit and this FD. Always add $(C, D) \rightarrow (a_0, \dots, a_{n-1})$. Each FD side uses attributes in A , including at the empty-universe boundary.

The forward map indexes arbitrary input labels by position. The decoder reads X from a positive target output and returns the original labels u_i for which $a_i \in X$, in input order. On NO-SOLUTION it returns NO-SOLUTION. The executable decoder also checks the bound and every family set.

4. Correctness

Lemma 1 (threshold). Before both C and D are present, $t_{i,j}$ is derivable from $X \subseteq R$ exactly when X contains at least j of $a_0, \dots, a_{i-1}$.

Proof. For $j = 1$, Equation 2 starts the count and Equation 3 preserves it. For $j > 1$, Equation 4 reaches j when the previous positions reach $j - 1$ and the last position is selected. These are all incoming rules for $t_{i,j}$. Induction on i proves the claim. Only the FD requiring both markers can add element attributes, so it cannot affect this phase. $\square$

For $X \subseteq R$, let $H(X)$ contain the original element u_i exactly when $a_i \in X$. Before both markers are present, an incidence attribute b_j is derivable exactly when $H(X)$ meets E_j . Thus C is newly derivable exactly when $H(X)$ hits every set. When $k < n$, Lemma 1 says that $t_{n,k+1}$ is derivable exactly when $|H(X)| > k$.

Lemma 2 (partial closure). The closure inside R gains some but not all missing attributes exactly when X contains neither marker and $H(X)$ is a hitting set of size at most k . In this case it gains C and no other member of R .

Proof. If either marker belongs to X , any newly derived member of R completes the marker pair, after which all element attributes follow. If neither marker belongs to X , a nonhitting selection derives neither marker. A hitting selection derives C ; when it has more than k elements, Lemma 1 also derives D , followed by every element attribute. A hitting selection within the bound derives C but not D . When $k \geq n$, every selection is within the bound and no threshold FD exists. $\square$

Theorem 1 (all-output reduction). For every legal source input and every valid target output, G returns a valid Hitting Set output.

Proof. Given a hitting set H of size at most k , choose X to contain precisely the attributes representing its elements. Lemma 2 makes (X, C, D) a valid target witness. Conversely, every valid target triple requires a partial closure inside R . Lemma 2 makes $H(X)$ a bounded hitting set, which G returns using the original element labels. This does not depend on the chosen y, z . A valid target NO-SOLUTION means no target witness exists; by the forward implication, no bounded hitting set exists, so source NO-SOLUTION is correct. Both output sets contain this special answer when no positive witness exists. $\square$

5. Complexity

Let L be the bit length of the explicit source input, including labels and binary k , and let s count element-set incidences. If $k < n$, then $q \leq n$, so the circuit has at most n^2 attributes and $O(n^2)$ FDs. Incidence FDs number s . Internal names use $O(\log(n + m + 2))$ bits. The target encoding has $O((n^2 + s + m) \log(n + m + 2)) \subseteq O(L^2 \log(L + 2))$ bits. Indexing labels, comparing binary k with n , and writing the target all take polynomial time in L .

The decoder scans the source instance and target output, checks the family, and returns at most n original labels. Their total encoding length is at most their occurrence in the source input. Its running time and output length are polynomial in $L + |w|$. Neither map calls a solver or retains state between invocations. Quadratic threshold growth can dominate target size; no practical large-instance runtime is claimed.

6. Scope

The rule gives a closure-checkable construction and exact all-output recovery for the stated bounded search task. It does not establish a new NP-completeness classification, the stronger 3NF restriction, or priority over inaccessible variants of the 1979 paper. The all-input claim rests on Lemmas 1–2 and Theorem 1; finite implementation checks appear in the appendix.

References

- [1] Philip A. Bernstein and Catriel Beeri. *An Algorithmic Approach to Normalization of Relational Database Schemas*. University of Toronto technical report, 1976, Part III §2, Theorem 8. Primary report.
- [2] Moshe Y. Vardi. *Fundamentals of Dependency Theory*. Chapter 5, Theorem 5.9. Survey.
- [3] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979, Appendix A4, SR29. Book scan.
- [4] Catriel Beeri and Philip A. Bernstein. “Computational Problems Related to the Design of Normal Form Relational Schemas.” *ACM Transactions on Database Systems* 4(1), 1979, pp. 30–59. DOI. Full theorem text not inspected.

A. Verification and reproducibility

The independent source oracle uses Z3 5.1.0 and exhaustive subset enumeration. They agree on all 125 fixed labels: 47 solvable and 78 unsolvable, from 106 seeded random and 19 edge cases. The prepared loop independently solves actual target instances by FD closure and checks 202 outputs. A separate Z3 encoding of staged target closure checks 52 instances and 77 outputs. A reviewer-owned check enumerates every target output on three explicitly labelled sources. These finite results address implementation behavior; they do not prove the theorem.

Reproduction requires Python 3.12.14 and uv 0.12.17. The lock installs Z3 Python binding 5.1.0.0, reporting solver version 5.1.0. The capability probe also found Kissat 4.0.4, Typst 0.15.1, Lean 4.34.0 and Lake 5.0.0; these were not needed for the reduction checks. CP-SAT and Mathlib were pending in the recorded probe. Run from the repository root:

```
uv sync --locked
uv run python campaigns/hitting-set-bcnf-violation/work/check.py --self-test
uv run python campaigns/hitting-set-bcnf-violation/work/check.py --candidate
campaigns/hitting-set-bcnf-violation/work/algorithm.py
uv run python campaigns/hitting-set-bcnf-violation/work/verify.py --candidate
campaigns/hitting-set-bcnf-violation/work/algorithm.py
uv run python campaigns/hitting-set-bcnf-violation/reviews/initial/check_domain.py
uv run python campaigns/hitting-set-bcnf-violation/reviews/label_repair/
check_labels.py
```

The forward command reads a source JSON object from standard input. Recovery reads the original source and a valid target answer. The two command modes are:

```
printf '%s\n' '{"universe":[2,4],"family":[[4]],"k":1}' | uv run python campaigns/
hitting-set-bcnf-violation/work/algorithm.py
printf '%s\n' '{"source":{"universe":[2,4],"family":[[4]],"k":1},"target_solution":
{"X":["a1"],"y":"C","z":"D"}}' | uv run python campaigns/hitting-set-bcnf-violation/
work/algorithm.py --extract
```

The latter returns {"set": [4]}. The fixed cases have at most six universe elements; fresh seeded cases have at most five. No solver timeout was used. Larger target instances and practical performance remain unmeasured.